

Systeme Intelligent de Traitement des Factures Electroniques

Systeme multi-agent LangGraph + LangChain
avec RAG multi-corpus et validation humaine

Etudiant :	ILYASS ELFATTAL
Filiere :	Master SDIA
Module :	Systemes Multi-Agents FC
Professeur :	Prof. RETAL Sara
GitHub :	github.com/elfetalelias/smart-einvoice-agent

1. Introduction et Problematique

Les entreprises marocaines traitent chaque jour de nombreuses factures fournisseurs. Ce processus est encore largement manuel : verification de la conformite fiscale, classification comptable selon le Plan Comptable Marocain (PCM), saisie des ecritures Debit/Credit. Cela prend du temps et genere des erreurs.

L'objectif de ce projet est d'automatiser ce traitement en construisant un systeme multi-agent intelligent qui orchestre plusieurs agents specialises, s'appuie sur un systeme RAG multi-corpus pour acceder a la legislation marocaine en vigueur, et garde le comptable humain au centre des decisions importantes.

2. Objectifs

- Extraire les donnees d'une facture (PDF, XML, image) avec un score de confiance
- Verifier la conformite fiscale selon le CGI 2026 et l'Article 145 (TVA, ICE, IF)
- Classifier la facture avec un code du Plan Comptable Marocain via RAG
- Generer l'écriture comptable Debit/Credit et verifier son equilibre
- Integrer trois points de validation humaine obligatoires dans le workflow
- Produire le journal comptable final structure
- Interface web avec traitement batch et export Excel/CSV (bonus)

3. Architecture du Systeme

Le systeme est construit autour de six agents qui travaillent ensemble de maniere sequentielle. Un superviseur central orchestre le flux via LangGraph - il lit l'etat courant de la facture et decide quel agent appeler ensuite.

L'extracteur prend en charge la lecture de la facture, que ce soit un PDF natif, un fichier XML ou une image, et produit des donnees structurees via un schema Pydantic. L'agent de conformite interroge ensuite le corpus RAG pour verifier les champs obligatoires selon l'Article 145 du CGI et la coherence de la TVA. Le classificateur comptable propose un code du Plan Comptable Marocain avec un score de confiance, toujours en s'appuyant sur le RAG. L'agent d'écriture genere alors l'écriture Debit/Credit en s'assurant que la somme est equilibree. Enfin, le journal comptable compile toutes les etapes traitees dans un document final.

Workflow LangGraph (graph/invoice_graph.py)

```
[extract_invoice]
|
[human_validation_1] <-- Validation #1 : donnees extraites
|
[check_compliance]
```

```
|  
[human_validation_2] <-- Validation #2 : conformite fiscale  
|  
[classify_accounting]  
|  
[human_validation_3] <-- Validation #3 : code comptable PCM  
|  
[generate_journal_entry]  
|  
[journal_comptable] --> END
```

Le choix d'une architecture hierarchique et sequentielle se justifie par la nature meme du traitement d'une facture : on ne peut pas classifier avant d'extraire, ni generer l'ecriture avant de verifier la conformite. Les trois noeuds de validation humaine bloquent le flux jusqu'a confirmation explicite de l'utilisateur.

4. RAG Multi-Corpus

Le systeme utilise trois corpus vectorises avec FAISS, chacun indexe separement. Un agent RAG central (`create_react_agent` de `langgraph.prebuilt`) recoit la question, choisit lui-meme le ou les outils pertinents, recupere les passages et formule une reponse en francais. Ce routing est implicite - le choix du corpus n'est jamais code en dur dans les agents metier.

Les trois corpus

- Corpus regles fiscales (`cgi-2026.pdf`) - TVA, ICE, IF, taux applicables
Tool : `search_regles_fiscales(query)`
- Corpus normes facturation (`cgi-2026.pdf` + `ubl_facturx.txt`) - Article 145 CGI, mentions obligatoires, sanctions, e-facture DGI
Tool : `search_normes_facturation(query)`
- Corpus Plan Comptable Marocain (PDF) - classes 2, 3, 4, 6, 7, codes comptables
Tool : `search_plan_comptable(query)`

Routing implicite par LLM

Tous les agents metier appellent `retrieve_context()` definie dans `rag/rag_router_agent.py`. Cette fonction construit l'agent RAG, lui envoie la question sous forme de message, et retourne le contenu du dernier message de la reponse. Le LLM choisit automatiquement le bon outil selon le contexte de la question posee.

5. Human-in-the-Loop

Trois points de controle sont integres dans le graphe LangGraph. A chaque point, le workflow attend une action explicite de l'utilisateur.

Validation #1 - Donnees extraites

L'interface affiche les donnees extraites (fournisseur, montants, ICE, IF, lignes). L'utilisateur peut corriger les montants avant de valider. Actions : Accepter / Annuler.

Validation #2 - Conformite fiscale

Les avertissements sont affiches avec leur niveau (CRITIQUE, MAJEUR, MINEUR) et la reference legale associee (ex : Article 145 CGI). Actions : Confirmer / Ignorer les avertissements et continuer.

Validation #3 - Classification comptable

Le code PCM propose est affiche avec sa justification et son score de confiance. Si la confiance est insuffisante (sous le seuil configure), un avertissement s'affiche. L'utilisateur peut corriger le code avant de generer l'ecriture.

6. Interface Utilisateur (Bonus)

L'interface est developpee avec Streamlit (app.py). Elle permet de traiter plusieurs factures en parallele via ThreadPoolExecutor.

- Etape 1 - Telechargement et extraction (PDF, XML, image, TXT)
- Etape 2 - Validation des donnees extraites avec correction possible
- Etape 3 - Resultats de conformite fiscale avec niveaux d'avertissement
- Etape 4 - Classification PCM avec correction manuelle du code
- Etape 5 - Journal comptable complet + export Excel (XLSX) et CSV

Le traitement batch utilise jusqu'a 5 workers en parallele. Chaque worker appelle directement les noeuds LangGraph (graph/nodes.py) de maniere independante par facture, ce qui permet de traiter plusieurs factures simultanement.

7. Stack Technique

- Python 3.11+
- LangChain - agents, tools, ChatPromptTemplate, with_structured_output
- LangGraph - StateGraph, conditional_edges, TypedDict InvoiceState
- OpenAI GPT-4o-mini - extraction, conformite, classification, journal
- FAISS - index vectoriels pour les trois corpus RAG
- Pydantic v2 - schemas structures (InvoiceData, ComplianceResult, AccountingCode, JournalEntry)
- Streamlit - interface web avec traitement batch
- pytest + pytest-cov - 93 tests, couverture 88%
- uv - gestionnaire de dependances (pyproject.toml + uv.lock)

8. Structure du Projet

Le projet est organise en modules separes selon les responsabilites. Le dossier agents/ contient les six agents specialises. Le dossier graph/ regroupe la logique LangGraph : l'etat partage (InvoiceState), le graphe (invoice_graph.py) et les noeuds (nodes.py). Les tools LangChain (lecteurs PDF/XML et retrievers FAISS) sont dans tools/. Le systeme RAG est isole dans rag/ avec ses corpus dans rag/corpora/. Les prompts de chaque agent sont stockes en fichiers .txt dans prompts/. La configuration (LLM, settings) est dans config/, les schemas Pydantic dans schemas/. L'interface Streamlit est dans app.py, les tests dans tests/ (93 tests, 88% couverture).

9. Installation

```
git clone https://github.com/elfetalelias/smart-einvoice-agent.git
cd smart-einvoice-agent

uv venv
```

```
source .venv/bin/activate
uv sync

cp .env.example .env
# Ajouter OPENAI_API_KEY dans .env

uv run python rag/ingest.py    # indexer les corpus RAG
uv run streamlit run app.py    # lancer l'application
```

10. Tests

93 tests, couverture globale 88%. Les agents metier sont couverts a 100%.

```
uv run pytest tests/ -v
uv run pytest tests/ --cov=. --cov-report=term-missing
```

- test_extractor_agent.py - schemas InvoiceData, extraction XML/PDF
- test_compliance_agent.py - coherence TVA, taux valides marocains
- test_accounting_classifier.py - classification PCM, seuil de confiance
- test_agents_additional.py - compliance, classifieur, journal, nodes LangGraph
- test_coverage_boost.py - couverture interne agents + RAG + tools
- test_invoice_graph.py - graphe LangGraph complet
- test_rag_retrievers.py - retrievers FAISS

11. Conclusion

Ce projet implemente un systeme multi-agent complet pour le traitement des factures electroniques marocaines. Les quatre points du projet de fin de module sont couverts :

- Workflow agentique : graphe LangGraph avec 8 noeuds et supervision hierarchique
- RAG multi-corpus : 3 corpus, 3 retrievers, routing implicite par LLM (create_react_agent)
- Human-in-the-Loop : 3 points de validation avec correction possible a chaque etape
- Interface web (bonus) : Streamlit avec traitement batch, export Excel/CSV

Les bonnes pratiques du cours sont respectees : separation agents/tools/graph/config, gestion des prompts dans prompts/, pas de melange LangChain/LangGraph sans logique claire, README complet, reproductibilite via uv.lock, et couverture de tests a 88%.

Lien GitHub

<https://github.com/elfetalelias/smart-einvoice-agent>